Module Guide for Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

November 14, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
Software Engineering	Explanation of program name
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 3

5 Module Hierarchy **4**

6 Connection Between Requirements and Design **5**

7 Module Decomposition **6**
7.1 Hardware Hiding Modules (M1) 6
7.2 Behaviour-Hiding Module 6
7.2.1 Input Format Module (M??) 6
7.2.2 Etc. 7
7.3 Software Decision Module 7
7.3.1 Etc. 7

8 Traceability Matrix **7**

9 Use Hierarchy Between Modules **8**

10 User Interfaces **9**

11 Design of Communication Protocols **9**

12 Database Design **9**

13 Timeline **9**

14 Generative AI Use Disclosure **9**
14.1 AI Tools Used 9
14.2 How and Where Used 9
14.3 Verification of AI-Generated Content 10

List of Tables

1 Module Hierarchy 5

2 Trace Between Requirements and Modules 7
3 Trace Between Anticipated Changes and Modules 8

List of Figures

1 Use hierarchy among modules 8

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable for the Troitsky Toolbox, which will be maintained by successive members of the club's software subteam as competition rules change each year and new tools are added.

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New software subteam members: This document can be a guide for a new member of the club's software subteam taking over maintenance of the Toolbox to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the anticipated changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the

completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules. Section 10 presents the design of the user interfaces. Section 11 describes the design of the communication protocols. Section 12 summarizes the database design. Section 13 outlines the schedule of tasks and the team members responsible for them. Finally, Section 14 discloses the use of generative AI tools in the preparation of this document.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adopted here is called design for change. Some anticipated changes affect more than one module; these cases are identified in the traceability matrix in Section 8.

AC1: The hosting platform and deployment configuration of the system. This may change from a cloud service to a university server depending on the club's budget and the servers available to it.

AC2: The competition rule set. The rules are revised and republished before each year's competition.

AC3: The library used to read STEP files and the club conventions for how bridge models are exported. The library may be replaced if it cannot read all club exports, and the export conventions may be refined as teams adopt them.

AC4: The structural analysis method and the set of performance metrics computed for a bridge. These will be iterated on to improve the accuracy of predictions, and new metrics will be added as the club identifies what is useful to teams.

AC5: The template for material test files and the method used to derive material properties from test results. These may change as the club adjusts its testing procedures or starts using new materials and suppliers.

AC6: The layout and design of the Toolbox Web App, including the library used to display 3D models. This will likely change based on user feedback to improve usability.

AC7: The set of tools offered by the Toolbox. Future software subteam members are expected to add new tools as the club's needs grow.

AC8: The database technology, schema, and backup strategy. The schema will grow as new tools and types of content are added, and the technology may change with the hosting platform.

AC9: The authentication mechanism used to sign members in. This may be replaced by an external sign-in provider, such as McMaster single sign-on, to simplify account management.

AC10: The structure and templates of handoff documents. Executives are expected to refine these each year based on what incoming members found useful.

AC11: The search strategy used to find content in the Knowledge Repository. This may need to change to keep searches fast as the amount of stored content grows.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decisions should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: keyboard, mouse, and file upload; Output: screen and downloaded files). Members interact with the system only through a web browser on a computer.

UC2: Delivery as a web application for desktop browsers. Mobile devices are not supported, as the main tasks, such as uploading CAD models, are done on a computer.

UC3: The broker-based architecture, in which all communication between services passes through the Toolbox Broker. This is fundamental to letting tools be added without modifying existing services.

UC4: The back-end technology stack of C# on .NET. Changing it would require rewriting most of the system, and it was chosen to be maintainable by future subteam members.

UC5: STEP as the only accepted bridge model format. STEP is supported by all the modelling software the club's teams currently use.

UC6: The overall format of the Troitsky competition. Individual rules change each year, but bridges are expected to remain built from the same materials and tested the same way.

UC7: The club roles on which access control is based. The club's organizational structure is not expected to change often.

UC8: Current club members as the only users of the system's services. Past members and the public have access only to the public landing page.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: Hardware-Hiding Module

M2: Web App Layout Module

M3: Toolbox Broker Module

M4: Authorization Rules Module

M5: Member Registry Module

M6: Repository Content Module

M7: Model Import Module

M8: Model Library Module

M9: Performance Metrics Module

M10: Rule Set Module

M11: Compliance Check Module

M12: Material Test Records Module

M13: Material Properties Module

M14: Handoff Documents Module

M15: Authentication Module

M16: Persistence Module

M17: Search Module

M18: STEP Parsing Module

M19: Structural Solver Module

M20: 3D Viewer Module

Level 1	Level 2
Hardware-Hiding Module	
Behaviour-Hiding Module	Web App Layout Module
	Toolbox Broker Module
	Authorization Rules Module
	Member Registry Module
	Repository Content Module
	Model Import Module
	Model Library Module
	Performance Metrics Module
	Rule Set Module
	Compliance Check Module
	Material Test Records Module
	Material Properties Module
	Handoff Documents Module
Software Decision Module	Authentication Module
	Persistence Module
	Search Module
	STEP Parsing Module
	Structural Solver Module
	3D Viewer Module

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

[The intention of this section is to document decisions that are made "between" the requirements and the design. To satisfy some requirements, design decisions need to be made. Rather than make these decisions implicit, they are explicitly recorded here. For instance, if a program has security requirements, a specific design decision may be made to satisfy those requirements with a password. —SS]

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Software Engineering* means the module will be implemented by the Software Engineering software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules (M1)

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Input Format Module (M??)

Secrets: The format and structure of the input data.

Services: Converts the input data into the data structure used by the input parameters module.

Implemented By: [Your Program Name Here]

Type of Module: [Record, Library, Abstract Object, or Abstract Data Type] [Information to include for leaf modules in the decomposition by secrets tree.]

7.2.2 Etc.

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Etc.

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	**Modules**
R1	M1, M??, M??, M??
R2	M??, M??
R3	M??
R4	M??, M??
R5	M??, M??, M??, M??, M19, M??
R6	M??, M??, M??, M??, M19, M??
R7	M??, M??, M??, M??, M??
R8	M??, M??, M??, M??, M??
R9	M??
R10	M??, M??, M??
R11	M??, M??, M??, M??

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M1
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M19
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

[The uses relation is not a data flow diagram. In the code there will often be an import statement in module A when it directly uses module B. Module B provides the services that module A needs. The code for module A needs to be able to see these services (hence the import statement). Since the uses relation is transitive, there is a use relation without an import, but the arrows in the diagram typically correspond to the presence of import statement. —SS]

[If module A uses module B, the arrow is directed from A to B. —SS]

Figure 1: Use hierarchy among modules

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

N/A. The Troitsky Toolbox is a web application with no custom hardware components, so there is no hardware-software communication protocol to design.

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.